PRINCETON UNIVERSITY PRESS

Chapter Title: PERCOLATION

Book Title: Natural Complexity

Book Subtitle: A Modeling Handbook

Book Author(s): Paul Charbonneau

Published by: Princeton University Press

Stable URL: https://www.jstor.org/stable/j.ctt1vwmgzt.7

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at https://about.jstor.org/terms

Princeton University Press is collaborating with JSTOR to digitize, preserve and extend access to *Natural Complexity*

JSTOR

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

4

PERCOLATION

We saw in the preceding two chapters that rule-based growth, whether in CAs or through DLA, can lead to the buildup of complex structures, sometimes exhibiting fractal geometry. We now examine another lattice-based system where similarly complex structures can arise from pure randomness. *Percolation* usually refers to the passage of liquid through a porous or granular medium. In its more abstract form, as developed in this chapter, it has become an exemplar of *criticality*, a concept in statistical physics related to *phase transitions*. An iconic example of the latter is liquid water boiling into water vapor, or freezing into ice.

Superficially, the simple lattice-based model introduced in this chapter bears no relation whatsoever to boiling water or to the flow of fluids through porous media. Yet it does capture the essence of the critical behavior characterizing these systems; such is the power of physical and mathematical abstraction.

4.1 Percolation in One Dimension

Consider a 1-D lattice of length N, i.e., a chain of N nodes each connected to its immediate right and left neighbors, with the exception of the two nodes at the ends of the lattice, which have only one neighbor. Figure 4.1 shows an $N = 64$ example. Each node has a probability p of being occupied (with, of course,

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

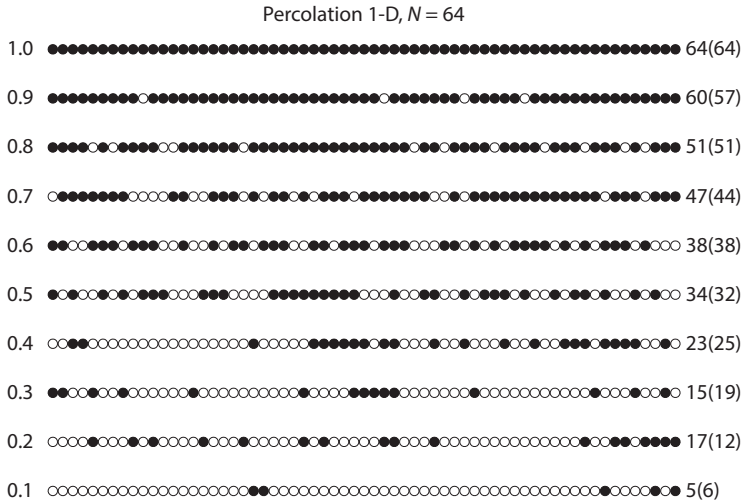

Figure 4.1. Percolation lattices in one spatial dimension. Each line represents an $N = 64$ lattice, with occupied nodes in black and empty nodes left as open circles. The occupation probability p increases from bottom to top in steps of 0.1, as indicated on the left. The number of occupied nodes is listed on the right, followed by the value $p \times N$ expected statistically, within parentheses.

$0 \leq p \leq 1$). This *occupation probability* is the same for all nodes, and is independent of neighboring nodes being empty or occupied; in other words, each node is *statistically independent* of all others on the lattice. For a very large lattice ($N \to \infty$), the expected number of occupied nodes tends toward pN, but at any finite N, deviations from this expected value are anticipated, and may be substantial for small N. This is indeed the case in figure 4.1, where the $p = 0.3$ lattice here contains *fewer* occupied nodes than at $p = 0.2$.

If p is small, only a few nodes on the lattice will be occupied, and most will have empty nearest-neighbor nodes. But as p is increased, the likelihood of having neighboring nodes occupied

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

also increases. Define a *cluster* as a set of contiguous occupied nodes, delineated by one empty node at each end. With p the probability of a node being occupied, then $1 - p$ is the probability of a node being empty. The probability of having at least one cluster of length s is thus

$$(1-p) \times \underbrace{p \times p \times p \times \cdots}_{s \text{ times}} \times (1-p) = p^s (1-p)^2. \quad (4.1)$$

This expression tends toward zero for very large clusters ($s \to \infty$), even in the limit $p \to 1$. This reflects the fact that one empty node somewhere is enough to "break" a cluster that is otherwise of length $s \to \infty$. Nonetheless, at some finite N, the probability of having a cluster of size s increases with p, as one would have expected.

Let s_k measure the size, i.e., the number of occupied nodes, for the kth cluster on the lattice, and denote by S the size of the largest such cluster:[1]

$$S = \max(s_k), \quad k = 0, 1, 2, 3, \ldots. \quad (4.2)$$

Consider now what happens as p is gradually increased. As long as relatively few nodes are occupied, one may expect that existing clusters will grow by "tacking" on a new occupied node at one of their extremities. The probability of this happening increases linearly with p, so one would expect $S \propto p$ for p small. This expectation is borne out in figure 4.1: S grows from 2 to 4 to 5 to 6 as p increases from 0.1 to 0.2 to 0.3 to 0.4. However, once a substantial fraction of lattice nodes are occupied, many clusters of significant size exist on the lattice, and a new growth process emerges: fusion of two preexisting clusters separated by one empty node, once that node becomes occupied. As p continues to increase and the lattice fills up, fusion of ever larger clusters

[1] To be consistent with Python's array indexing convention (see appendix A), the K clusters on the lattice are numbered from 0 to $K - 1$. Sorry!

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

becomes increasingly frequent, and leads to very rapid growth of S. It can be shown that in the limit $N \to \infty$, the size of the largest cluster grows according to

$$\lim_{N\to\infty} S = \frac{1+p}{1-p}. \tag{4.3}$$

This indicates that the size of the largest cluster tends to infinity in the limit $p \to 1$. In other words, the largest clusters reach sizes comparable to that of the whole system. The numerical value of p at which this happens is called the *percolation threshold*, hereafter denoted p_c. For 1-D lattices, $p_c = 1$ for the very simple reason that only one empty lattice node is enough to "break" the infinite cluster. This conclusion would have been easy to anticipate without all this probabilistic mumbo jumbo, but it was important to go through it nonetheless, because things become a lot trickier—and complex!—for lattices in more than one spatial dimension.

4.2 Percolation in Two Dimensions

Let's move to 2-D lattices, and see how much of what we learned in one dimension carries over. In what follows we restrict ourselves to regular Cartesian lattices with top/down/right/left nearest-neighbor connectivity. Each node on the lattice is identified by a pair of integers (i, j) flagging its "vertical" and "horizontal" locations, respectively (if needed, see appendix D for more on lattice definition and notation). Except for nodes located at the lattice boundaries, the 4 nearest neighbors of node (i, j) are

$$\underbrace{(i-1, j)}_{\text{top}}, \quad \underbrace{(i+1, j)}_{\text{down}}, \quad \underbrace{(i, j+1)}_{\text{right}}, \quad \underbrace{(i, j-1)}_{\text{left}}. \tag{4.4}$$

Here is a small source code in the Python programming language that defines such a 2-D lattice of size 128×128, and fills it with occupation probability $p = 0.59$.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

Figure 4.2. Two-dimensional regular Cartesian lattices of size $N \times N = 64 \times 64$, with occupation probabilities $p = 0.25$, 0.5, and 0.75. Occupied nodes are filled in black, and empty nodes are white.

```
# CREATES AND FILLS A 2-D CARTESIAN PERCOLATION LATTICE
import numpy as np
import matplotlib.pyplot as plt
#-----------------------------------------------------------------------
N=128                                # lattice size
p=0.59                               # occupation probability
np.random.seed(1234)                 # seed for random number generator
#-----------------------------------------------------------------------
lattice=np.zeros([N,N],dtype='int')  # 2-D lattice initialized to zero

for i in range(0,N):                 # lattice loops
   for j in range(0,N):
      if np.random.uniform() < p:    # occupy this node
         lattice[i,j]=1
plt.imshow(lattice,interpolation="nearest")
plt.show()                           # display lattice
# END
```

Note that the value 1 is used to identify an occupied node, empty nodes being set to 0. Clusters are now defined as groups of contiguous occupied nodes separated from other clusters or single occupied nodes by empty nodes. The percolation threshold is now defined as the value of p at which the largest cluster spans the whole lattice, in the sense that it "connects" one lattice boundary to its counterpart on the facing boundary.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

Figure 4.2 shows three examples of 2-D regular Cartesian lattices of size $N \times N = 64 \times 64$, with occupation probabilities $p = 0.25$, 0.50, and 0.75. At $p = 0.25$, the lattice contains a large number of small clusters or isolated occupied nodes. Their spatial distribution is random but statistically uniform. It is quite clear here that no single cluster spans the whole lattice, so we are obviously below the percolation threshold. At $p = 0.75$, the lattice looks like a porous object, a bit like a sponge, containing many small holes again distributed randomly but in a statistically uniform manner. Here, one single, very large cluster fills the lattice and contains the majority of occupied nodes. This indicates that we are *beyond* the percolation threshold. The $p = 0.5$ case is more ambiguous, at least visually. Are we seeing a dense clump of large clusters, or a highly fragmented solid structure? That lattice would have to be studied carefully to verify whether or not one cluster extends from one end of the lattice to another. But figure 4.2 already allows us to draw one interesting conclusion: unlike in the 1-D case, here the percolation threshold $p_c < 1$. This is because in two spatial dimensions, an empty node can be bypassed.

4.3 Cluster Sizes

If building a 2-D percolation lattice can be done in a few lines of Python code, identifying and sizing clusters is a much more complex endeavor. There are many algorithms available to do this, and the bibliography at the end of this chapter includes a few good references for those wishing to delve into the state of the art. The algorithm introduced in what follows is far from the most efficient, but it is relatively easy to code and conceptually simple to understand.

Imagine tagging an occupied node with a specific color, say green; starting from this newly colored node, color green all occupied nodes that are nearest neighbors, and then their nearest neighbors, and so on until no uncolored nearest neighbors

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

are found. Then move to the next as-yet-uncolored, occupied node, and repeat this process with a new color tag. Continue in this manner until no uncolored, occupied node is left on the lattice, and each cluster will end up tagged with a unique color.

The Python code in figure 4.3 is a direct implementation of this simple algorithm. This user-defined function could be called directly, at the end of the small code presented at the beginning of this section, and the clusters plotted, through the following instruction:

```
n_cluster,size_cluster,tag_cluster,map_cluster=findcluster(N,lattice)
```

Algorithmically, this function operates along the lines described above:

1. The first step is to copy the `N`×`N` lattice into a working array `map_cluster` of size (`N`+2) × (`N`+2), thus leaving a padding of unoccupied ghost nodes (value = `0`) along its perimeter. This is carried out on line 9, through the implicit looping allowed by the "`i1:i2`" array index syntax in Python, which means "access elements starting at index value `i1` up to position `i2` (meaning, index `i2-1`(!); see appendix A.2 for more on this if needed). This will allow the nearest-neighbor check to be carried out for all nodes using the same relative template, much as in the DLA code of figure 3.1; otherwise, boundary nodes would need to be treated differently, increasing coding complexity. See appendix D.1 for more on the use of ghost nodes.
2. The algorithm is built on two nested outer loops, each running over one dimension of the lattice (lines 12–13), scanning it line by line.
3. At each node scanned within the outer loops, a test verifies whether the node is occupied (value `1`) and not

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

```
1  # FUNCTION FINDCLUSTER: TAGS AND PLOTS PERCOLATION CLUSTERS ON A 2D LATTICE
2  #    lattice is supposed of size NxN with nodal value 1 indicating an
3  #    occupied node and a value 0 for an empty node
4  def findcluster(N,lattice):
5     dx,dy=np.array([-1,0,1,0]),np.array([0,-1,0,1]) # Template arrays
6     size_cluster=np.zeros(N*N/2,dtype='int')        # Cluster size array
7     tag_cluster =np.zeros(N*N/2,dtype='int')        # Cluster tag array
8     map_cluster =np.zeros([N+2,N+2],dtype='int')    # Cluster map array
9     map_cluster[1:N+1,1:N+1]=lattice[:,:]           # Pad lattice with zeros
10    n_cluster,iic=0,100                             # Counter, first tag
11
12    for j in range(1,N+1):                          # Outer lattice scan
13       for k in range(1,N+1):
14          size,add_to_size=0,0                      # Initialize counters
15
16          if map_cluster[j,k] == 1:                 # Initiate new tagging
17             map_cluster[j,k]=iic                   # New cluster tag
18             size+=1                                # First node of cluster
19             add_to_size+=1
20
21             while( add_to_size > 0)                # Tagging in progress
22                add_to_size=0
23                j1,j2=j,min(N,j+size)               # Range of inner scan
24                k1,k2=max(1,k-size),min(N,k+size)
25                for jj in range(j1,j2+1):           # Inner lattice scan
26                   for kk in range(k1,k2+1):
27                      if map_cluster[jj,kk] == 1:   # Untagged occupied node
28                         if iic in map_cluster[jj+dx[:],kk+dy[:]]:
29                            map_cluster[jj,kk]=iic  # assign tag to node
30                            size+=1
31                            add_to_size+=1
32             # end of inner lattice scan
33             size_cluster[n_cluster]=size           # Size of this cluster
34             tag_cluster[n_cluster] =iic            # Tag for this cluster
35             print("cluster tag {}, size {}.".format(iic,size))
36             iic=np.random.random_integers(10,250)  # Set up next tag
37             n_cluster+=1                           # Increment counter
38          # end of this tagging
39
40    # end of outer lattice scan; display clusters
41    plt.imshow(map_cluster,interpolation="nearest") # Display clusters
42    plt.show()
43    return n_cluster,size_cluster,tag_cluster,map_cluster
44 # END FUNCTION FINDCLUSTER
```

Figure 4.3. A user-defined Python function for identifying and tagging individual clusters in a 2-D percolation lattice with 4-neighbor connectivity. The matplotlib instructions on lines 41–42 display the clusters essentially as in figure 4.5.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

yet assigned to a cluster (line 16). If so, a unique numerical tag (variable `iic`) is assigned to it (line 17).

4. If and only if a new tag has been generated, a new "inner" lattice scan is initiated (line 21–32). Each occupied node having a nearest neighbor with identifier `iic` is tagged with that same identifier (lines 27–29). A `while` loop construct (starting at line 21) ensures that the inner lattice scan is repeated until no untagged, occupied node is found with an `iic`-tagged neighbor in the course of a complete scan.
5. By the design of the algorithm, the cluster being tagged can enlarge only by one nodal distance horizontally and/or vertically from its starting node at each iteration of the `while` loop. Consequently, the inner lattice scan spans an increasing range of nodes with each iteration (lines 23–24), with the use of `min/max` to avoid out-of-bounds array indexing on the array `map_cluster`. Note also that the order in which the lattice is scanned implies that all nodes with index `jj < j` have already been tagged, so that the range of the inner loop on line 25 begins at `j1=j`.
6. The nearest-neighbor check uses the two 4-neighbor template arrays `dx` and `dy`, verifying whether any nearest neighbor has already been tagged with the value `iic` (line 28). Note here the use of the Python-specific construct `if iic in...`, which means "if value `iic` is found in the set of array elements following"; if needed, see appendix A.5 for an equivalent set of Python instructions using only simple `for` and `if` instructions.
7. At the end of the inner lattice scan, the number of nodes tagged with value `iic` is stored in the array `size_cluster` (line 33), and the outer lattice scan resumes from where it had been interrupted, until a

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

new untagged, occupied node is located, in which case step 3 begins anew, or until the outer scan reaches the end of the lattice.

8. At the end of the outer lattice scan, the integer variable `n_cluster` contains the number of clusters identified, the array `size_cluster` contains the size (measured in number of nodes) of each of these clusters, in the order of their tagging, and the array `tag_cluster` contains the corresponding numerical value of the tags. Nodal values in the lattice array `map_cluster` now contain, at occupied nodes, the tag value `iic` associated with each cluster, instead of the value `1` originally indicating an occupied node (as per line 29). These are the quantities returned by the function (line 43).
9. The `size_cluster` and `tag_cluster` arrays are assigned a length of $N^2/2$ (lines 6–7), which is equal to the largest number of clusters than can be fit on an $N \times N$ lattice, namely, clusters all of size `1` distributed as a checkerboard pattern.

Figure 4.4 illustrates the operation of this cluster-tagging algorithm, here for a small 16×16 lattice at occupation probability $p = 0.58$. In the top-left frame, five clusters have already been tagged, as indicated by distinct colors, and the 12 frames cover successive tagging steps (iterations of the `while` loop) within the outer lattice loop, starting from a untagged, occupied node at the upper left (in green).

This algorithm is (relatively) easy to code but inefficient in a number of ways, notably the fact that the outer and inner sets of loops spend a lot of time revisiting nodes that are unoccupied or have already been tagged. A more efficient approach, relatively straightforward to code in Python, would be to first build a *list* of occupied nodes, and replace the two sets of loops at lines 12–13 and 25–26 by a single loop over elements of that list. Elements of

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

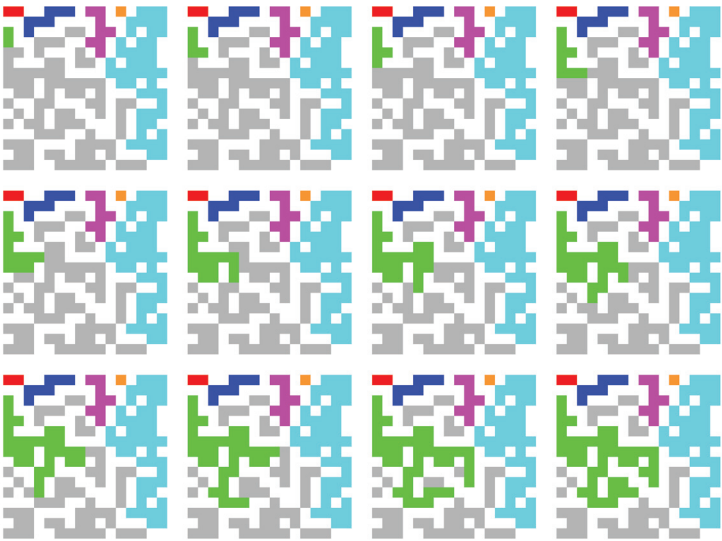

Figure 4.4. The cluster-tagging algorithm of figure 4.3 in action, here on a 16×16 lattice at $p = 0.58$. The nodal position $(0, 0)$ is at the top left in each frame. The outer and inner lattice scans proceed, line by line, from top to bottom (loop indices `j` and `jj`) in the code of figure 4.3, and from left to right in each line (loop indices `k` and `kk`). Colored squares correspond to occupied nodes already tagged to a cluster, while gray squares indicate as-yet-untagged occupied nodes, and white squares indicate empty nodes. In the top-left frame, a new tagging inner loop has just started at the left extremity of the third line of the lattice (nodal position $(j, k) = (2, 0)$), and the bottom-neighbor node has just been tagged (both in green). The next 11 frames show successive steps of the tagging process, each corresponding to an iteration of the `while` loop at line 21 in figure 4.3, the sequence being obvious. At the end of the tagging process (bottom right), a cluster of 44 nodes has been tagged "green." The outer lattice scan would now resume in the third column, with the node $(j, k) = (2, 1)$. The next tag would be initiated at node $(2, 6)$, for a cluster of size 2.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

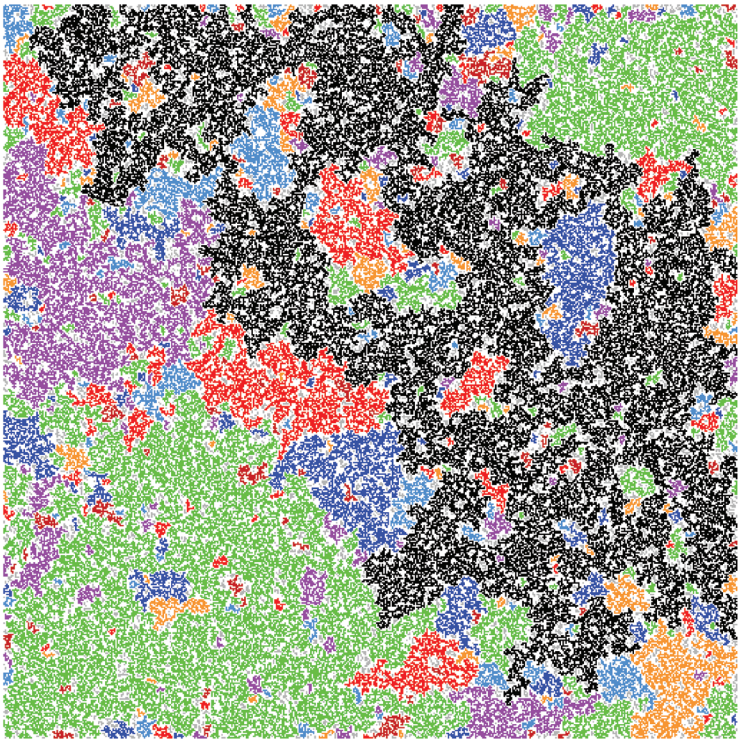

Figure 4.5. The 661 largest clusters on a 512×512 lattice at $p_c = 0.59$. Empty sites are left white, while gray indicates occupied nodes that are not part of one of the 661 largest clusters. The largest cluster, plotted in black, collects $S = 53,537$ of the 154,867 occupied nodes, and spans the whole lattice. Notice how holes in the larger clusters contain smaller clusters, themselves with holes containing even smaller clusters, and so on down to single occupied nodes. The matplotlib instructions at the end of the cluster-tagging function in figure 4.3 generate essentially this type of display.

the array `grid` are tagged as before, but nodes are then removed from the list as they are tagged.

Figure 4.5 shows the end result of the tagging algorithm of figure 4.3, here for a 512×512 lattice at $p = 0.59$. Upon completion of the tagging algorithm, locating and tracing the

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

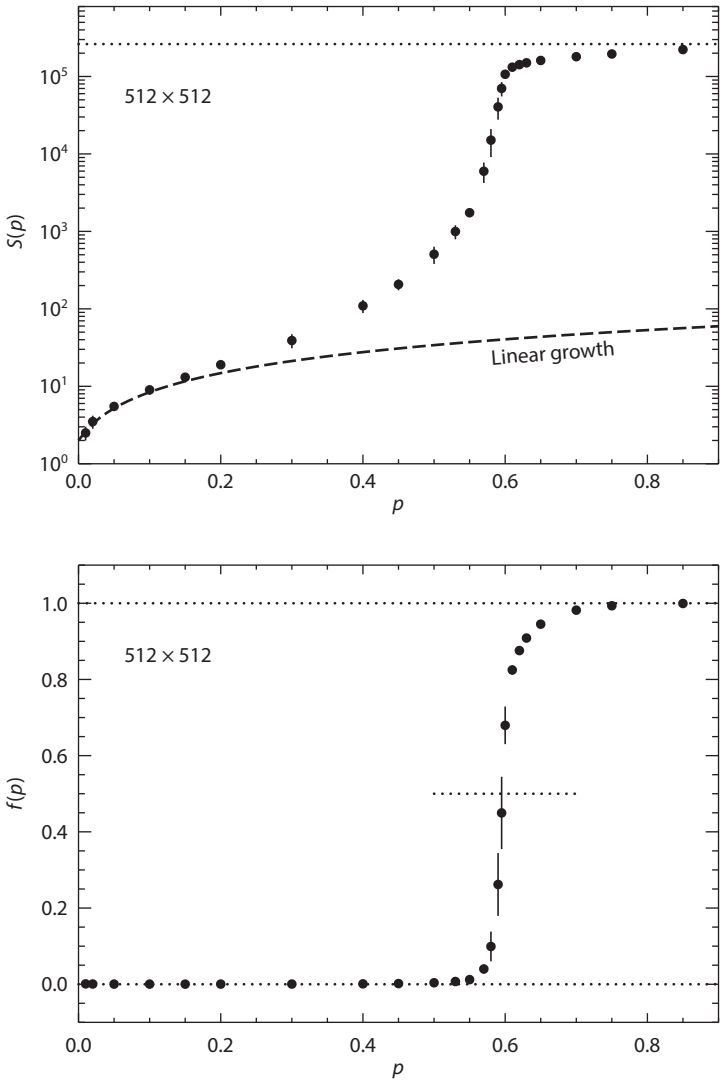

Figure 4.6. The top panel shows the growth of the largest cluster on a 512 × 512 lattice, as a function of the occupation probability p. Note the logarithmic vertical axis. Each solid dot is an average over 10 statistically independent realizations of the lattice at the same value of p, with the vertical line segments indicating the standard

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

largest cluster simply requires searching the array `size_cluster` for its largest element, retrieving the associated tag number from the array `tag_cluster`, and finally extracting the correspondingly numbered nodes from the cluster map array `map_cluster`. These jointly form the largest cluster, colored in black in figure 4.5. The top panel in figure 4.6 shows how the size S of that largest cluster increases with the occupation probability p, still for an $N \times N = 512 \times 512$ lattice. What is plotted is actually the mean largest cluster size $\langle S \rangle$, averaged over $M = 10$ realizations of the lattice at each value of p (solid dots), with the vertical line segments indicating the standard deviation σ_S about the mean:

$$\langle S \rangle = \frac{1}{M} \sum_{m=1}^{M} S_m, \quad \sigma_S = \left(\frac{1}{M} \sum_{m=1}^{M} (S_m - \langle S \rangle)^2 \right)^{1/2} \tag{4.5}$$

As when computing the fractal dimension of DLAs in the preceding chapter, such *ensemble averaging* is carried out to ensure that the plotted variation is representative, and not distorted by the idiosyncracies of a specific lattice configuration, each percolation lattice being as unique as the seed provided to the random number generator upon initialization (if needed, see appendix C.2 on this seed business). As expected, the size S of the largest cluster grows with the occupation probability.[2] For

[2]Note that throughout this chapter, the term "growth" is used even though it does not arise from the action of a dynamical process, such as in chapters 2 and 3.

Figure 4.6 (*Continued*). deviation σ_S about the the ensemble mean $\langle S \rangle$ (see equations (4.5)). The dashed curve corresponds to linear growth, and the dotted line indicates the largest cluster size possible on the lattice, here $512 \times 512 = 2.62 \times 10^5$. The bottom panel plots the same numerical results, but for S normalized by the total number of occupied nodes (see equation (4.6)), and with the vertical axis now linear rather than logarithmic.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

$p \lesssim 0.2$, linear growth (dashed curve) fits the numerical data tolerably well, but in the range $0.2 \lesssim p \lesssim 0.6$ growth has already become super-exponential (i.e., upward curvature in this log-lin plot). This reflects successive pairwise fusion of existing clusters, through the occupation of single nodes that had remained empty at lower p values. Sometimes, occupying just one more node is all it takes. The rapid saturation at $p \gtrsim 0.6$ is set by the size of the lattice, which limits here the largest cluster to a maximal size of 512×512 (dotted line).

The bottom panel in figure 4.6 shows the same results as in the top panel, except that now the size of the largest cluster has been normalized by the expected number of occupied nodes on the lattice at each value of p, i.e.,

$$F(p) = \frac{S(p)}{pN^2}. \tag{4.6}$$

This measures the *fraction* of occupied nodes belonging to the largest cluster, and it highlights something interesting: as long as $p \lesssim 0.5$, $F(p)$ remains close to zero, even though the absolute size of the largest cluster is growing significantly (cf. top panel, and its logarithmic vertical axis!). In other words, the largest cluster becomes bigger, but does not particularly stand out as compared to other clusters on the lattice. At the end of the range, $p \gtrsim 0.65$, the largest cluster includes almost all occupied nodes, which we expected already. But what is striking is the sharpness of the transition between these two regimes. Around $p = 0.55$, $F(p)$ grows very rapidly, already approaching saturation close to unity at $p \simeq 0.65$. Indeed, around $p = 0.6$, the growth of S appears to diverge, in the (calculus) sense that $\mathrm{d}F/\mathrm{d}p \to \infty$. The exact value of p at which this takes place defines the percolation threshold for this 2-D lattice. At this threshold, the largest cluster contains on average half of the occupied nodes:

$$S(p_c) = \tfrac{1}{2} p_c N^2. \tag{4.7}$$

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

For a 4-nearest-neighbor 2-D Cartesian lattice, the percolation threshold turns out to be at $p_c = 0.592746$. Unlike in the 1-D case, there exists no equivalent to equation (4.3), and the percolation threshold must be evaluated numerically.

Enough (for now) with the largest cluster, and let's turn to the population of all clusters on the lattice. This information is contained in the array `size_cluster` returned by the cluster-tagging code listed in figure 4.3. We now want to get a measure for the range of cluster sizes found in the lattice. To this end, the most useful mathematical object is a *probability density function* (hereafter, often abbreviated as PDF) of cluster sizes.

Mathematically, the PDF $f(s)$ is defined such that $f(s)\Delta s$ gives the probability of finding on the lattice a cluster of size between s and $s + \Delta s$. Figure 4.7 plots such PDFs of cluster sizes on an $N \times N = 512 \times 512$ lattice with $p = 0.3$ (red), 0.59 (green), and 0.7 (blue). In essence, these discrete PDFs thus measure, in each simulation, the frequencies of clusters having a size that falls within each of the histogram bins.[3] Like most PDFs to be encountered later in this book, the PDFs in figure 4.7 are plotted in so-called histogram mode, to emphasize their fundamentally discrete nature: a count of clusters in a given size range Δs is an integer number and characterizes a finite-sized range.

In the first case, $p = 0.3$, the PDF drops rapidly as s increases, reflecting the fact that the lattice is populated by small clusters (as in figure 4.2, left panel). At $p = 0.7$, one gigantic cluster contains nearly all occupied nodes (as in figure 4.2, right panel). This single supercluster accounts for the single blue histogram column at $s \simeq 2 \times 10^5$. The remaining clusters are small ones dispersed in

[3]Readers unfamiliar with this concept should really read appendix B before proceeding any further. Note also that the PDFs plotted in figure 4.7 are constructed using logarithmically constant bin sizes Δs, as described in appendix B.5.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

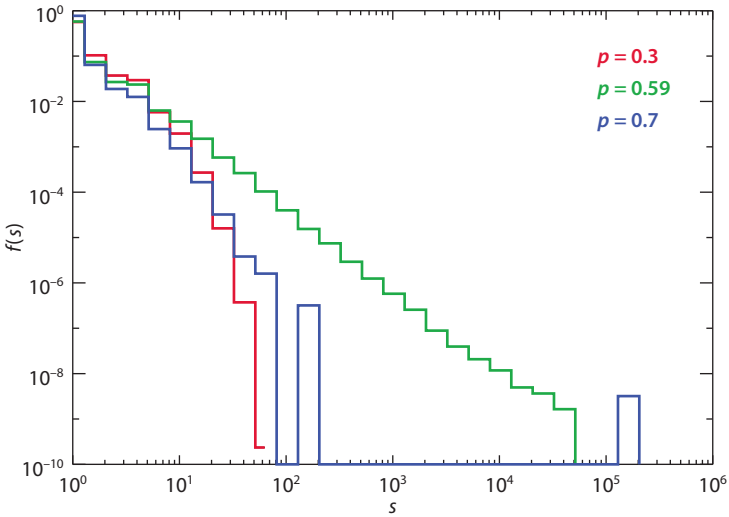

Figure 4.7. Probability density functions of cluster sizes on a 512×512 lattice for $p = 0.3$ (red), 0.59 (green), and 0.7 (blue). Note the similarity between the PDFs at $p = 0.3$ and $p = 0.7$, the crucial difference being the presence of the lone blue histogram bin at $s \simeq 2 \times 10^5$, corresponding to the single largest cluster covering most of the lattice at $p = 0.7$.

the cavities of the supercluster, and their PDF closely resembles that of clusters at $p = 0.3$. The case $p = 0.59$ is very close to the percolation threshold, and stands out in that its PDF takes the form of a power law spanning essentially the whole range of cluster sizes accessible on the lattice:

$$f(s) = f_0 s^{-\alpha}, \quad \alpha > 0, \tag{4.8}$$

here with $\alpha \simeq 1.85$. What is truly remarkable is that the numerical value of this exponent is independent of lattice size, as shown in figure 4.8. In going from small lattices of a few

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

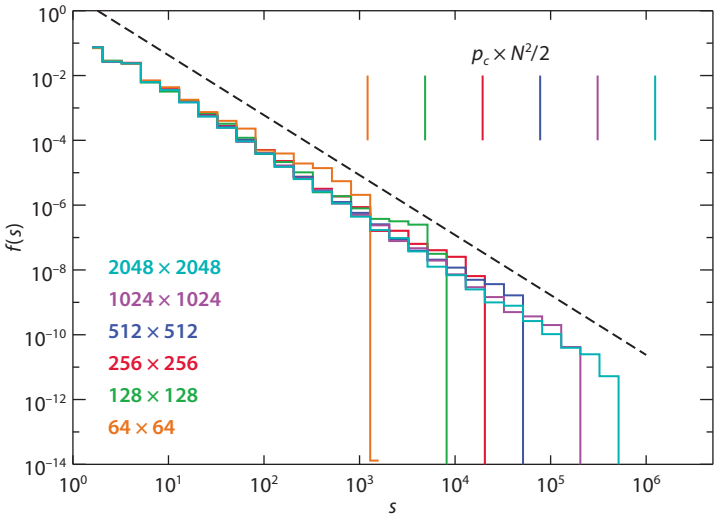

Figure 4.8. Cluster-size PDFs near the percolation threshold, for 10-member ensembles of statistically independent realizations of lattices ranging in size from $N \times N = 64 \times 64$ to 2048×2048, as color coded. The vertical tick marks in the upper right indicate the expected size ($S = p_c N^2/2$) for the largest cluster on each lattice, and the dashed line is drawn with a logarithmic slope of -1.85, which on such log-log plots gives the exponent α in equation (4.8) directly.

thousand nodes, to lattices in excess of a million nodes, the PDF retains the same power-law shape and logarithmic slope; all that changes is the extension of the distribution to ever larger sizes, the cutoff always occurring very near the expected size $p_c N^2/2$ for the largest cluster at the percolation threshold. The power-law index $\alpha = 1.85$ is said to be *universal* for this class of 2-D Cartesian lattices with 4-nearest-neighbor connectivity. These PDFs are again ensemble averages of 10 realizations of the percolation lattice at each value of occupation probability.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

Each PDF is constructed from combining cluster counts for all 10 realizations, and then the power-law index $\alpha = 1.85$ is calculated for this joint PDF.[4]

4.4 Fractal Clusters

A robust power-law PDF is indicative of scale invariance in the structure being measured. We have already encountered scale invariance in our discussion of fractal geometry in the preceding chapter. Could clusters on a lattice at the percolation threshold be fractal objects? Let us look into that.

Figure 4.9 displays the largest cluster found on a 512×512 lattice, for occupation probabilities ranging from $p = 0.57$ to 0.6, as labeled. The largest cluster is still significantly smaller than the lattice at $p = 0.57$, already spans it at $p = 0.59$, and fills it in a spongelike manner at $p = 0.6$. The shapes of these clusters are noteworthy. Close to the percolation threshold, the clusters are very filamentary and contain many large cavities, which contain smaller clusters also with cavities, also containing smaller clusters, and so on down to the scale of the lattice interval (see figure 4.5), in the same classical scale-invariant manner as in the DLA in figure 3.2. Clusters are indeed fractal objects, with a dimension somewhere between 1 and 2. Their fractal index varies with the occupation probability p, the numerical value being smallest at the percolation threshold. Because of their highly irregular shape, the fractal dimension of clusters is best computed using the box-counting method introduced in section 3.5.

4.5 Is It Really a Power Law?

Power-law PDFs pop up everywhere in measurements of "event sizes" in naturally occurring phenomena: for example, avalanches, forest fires, earthquakes, solar flares, to name but a few that

[4]Note that this is *not* the same as averaging the 10 power-law indices associated with the individual PDFs for each member of the ensemble, since a power law is a nonlinear function of its independent variable, which here is size occurrence frequency $f(s)$.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

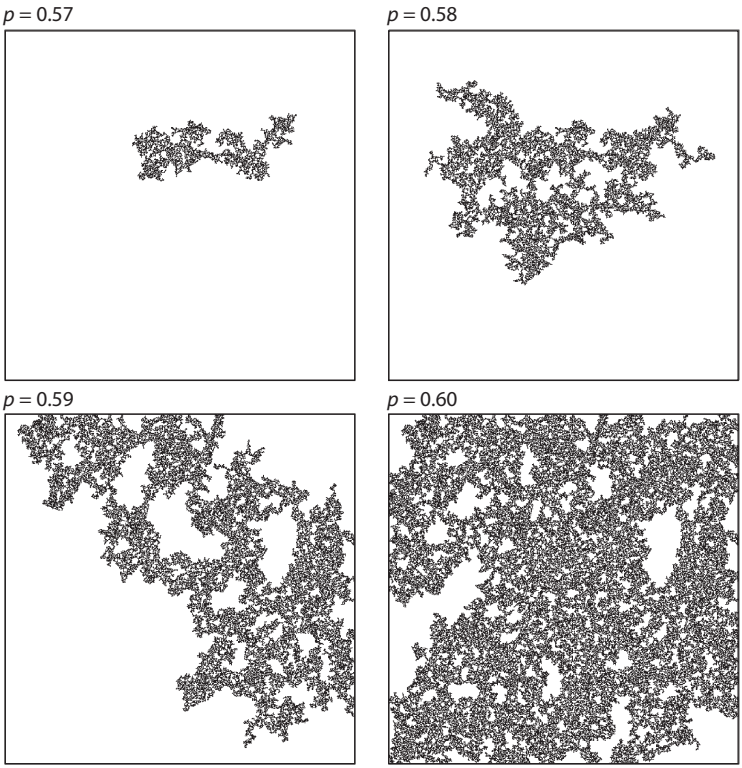

Figure 4.9. The largest clusters on a 512×512 lattice, for occupation probabilities $p = 0.57$, 0.58, 0.59, and 0.6. Over this very restricted range, a small increase in p leads to a pronounced increase in the size of the largest cluster.

will be encountered in subsequent chapters. The implied scale invariance holds important clues to the underlying dynamical processes driving these events, and consequently a reliable empirical determination of power-law form (4.8) and associated index α is important.

The power-law index $\alpha = 1.85$ characterizing the PDFs in figure 4.8 was obtained by a linear least-squares fit on the 2048×2048 PDF in the range $10 \leq s \leq 10^5$, with the fit carried out in the log-log plane and equal weight assigned to

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

each histogram bin. This very simple method has its limitations and must be used with proper caution. If the PDF extends over many orders of magnitude in the measured variable (here over five orders of magnitude in s) and is built from a great number of measured events (here over 10^6 for the 10-member ensemble), the inferred power-law index typically turns out to be fairly accurate; this is often no longer the case for steeper power-law PDFs, spanning only a few orders of magnitude and/or built from a smaller sample of measured events. Some robust statistical approaches have been designed, that allow reliable determination of power-law indices even under these circumstances. See appendix B.6 for more on these matters.

The fractal structure of percolation clusters will certainly not extend to the very smallest cluster sizes possible: a cluster of size 2 is definitely a line; so are one-third of clusters of size 3, the other two-thirds having the shape of 90-degree wedges; and so on. Scale invariance will surely break down before reaching the smallest cluster size, like it did when zooming in on the DLA aggregates (see figure 3.12). Likewise, the finite size of the percolation lattice will inevitably distort the shape of the largest clusters. This effect is clearly visible in figure 4.9 where, close to the percolation threshold ($p = 0.59$; bottom left), parts of the largest cluster are clearly deformed due to the presence of the lattice boundaries. Scale-invariant power-law behavior is thus expected to break down at the high end of the cluster-size distribution as well. This is why the fit in figure 4.8 is carried out using data in the range $10 \leq s \leq 10^5$ only.

4.6 Criticality

Let's summarize what we have learned so far about 2-D percolation. At and only at the percolation threshold p_c, the following holds:

1. The sizes of clusters are distributed as a power law.
2. The linear dimension of the largest cluster is $\simeq N$.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

3. The largest cluster collects a fraction $F = 0.5$ of all occupied nodes.
4. The growth rate of $\langle S \rangle$ diverges ($\mathrm{d}\langle S \rangle / \mathrm{d}p \to \infty$) in the limit $p \to p_c$.
5. The root-mean-squared (rms) deviation of the size of the largest cluster, relative to the mean value, is largest.
6. The fractal dimension of the largest cluster reaches its smallest numerical value.

These behaviors characterize what is known in statistical physics as a *critical system*. The operational defining characteristic of a critical system is its global extreme sensitivity to a small perturbation in the system. Phase transition in water is the typical example, whereby water at 100°C transits from liquid to vapor; there is no such thing as a pot of half-boiling water; either the whole pot is boiling, or it is not, and under so-called standard atmosphere conditions, the transition point is at exactly 100°C. A tiny fraction of a degree below 100, and the water is liquid; a tiny fraction of a degree above, and the water is already vapor. But at exactly 100°C, adding a tiny increment of heat will trigger the phase transition.[5]

The link with percolation is with the behavior of the largest cluster as a function of the occupation probability. When $p < p_c$, adding an occupied node will perhaps enlarge a cluster; when $p > p_c$, there is already a large cluster spanning the lattice, and adding one more occupied node to it will not change much. But at exactly $p = p_c$, adding a single node *may* connect two existing large clusters to generate a cluster spanning the whole lattice. If we think of the latter as a porous medium (occupied node = material, empty node = hole), the system goes suddenly from permeable to impermeable. If the lattice is viewed

[5]In the language of statistical physics, one would say that the correlation length of a perturbation becomes comparable to the size of the system.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

as some composite material made of electrically conducting grains (occupied nodes) embedded in a nonconducting matrix (empty nodes), then at the percolation threshold the system goes suddenly from nonconducting to electrically conducting. Other well-studied examples include magnetization at the Curie point, polymerization of colloidal liquids, and superfluidity in liquid helium, to name but a few.

In all these systems, critical behavior materializes when a *control parameter* is very finely tuned to a specific value—$p_c = 0.59274$ for percolation on a 4-neighbor 2-D Cartesian lattice; a temperature of 100°C for boiling water, etc.—by a mechanism external to the system. The need for such finely tuned external control may suggest that criticality is unlikely to develop spontaneously in natural systems, which are typically not subjected to finely tuned external control.

It turns out that many natural systems *can* reach a critical state autonomously, through the action of their own dynamics, and, as a matter of fact, the following chapter introduces one.

4.7 Exercises and Further Computational Explorations

1. Go back to take a look at figure 3.5; would you say this "lattice" is at the percolation threshold? Why?
2. This mathematical task is to show that in the regime of small p, the largest cluster on a 1-D lattice grows linearly with p; specifically,

$$\lim_{p \ll 1} S = \lim_{p \ll 1} \frac{1+p}{1-p} \simeq 1 + 2p. \tag{4.9}$$

3. Construct a series of 1-D percolation lattices of length $N = 128$, with occupation probability ranging from $p = 0.1$ to $p = 0.9$ in steps of 0.1, like in figure 4.1. For each value of p, construct 10 such lattices, each using a different seed for the random number generator controlling the loading of the lattice (see section 4.2

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

and appendix C). Now, for each p value, determine the mean number of occupied nodes, as averaged over the 10 realizations of the lattice, and compare it to the expected value pN. Then, calculate the mean size of the largest cluster $\langle S \rangle$ for each p, again averaged over your 10 lattice realizations, and plot this mean value as a function of p. Identify the value of p at which the growth process switches from single-node addition to cluster fusion.

4. Generate a 2-D 256×256 Cartesian percolation lattice at $p = 0.59$, following the procedure described in section 4.2, and use the code listed in figure 4.3 (or some equivalent) to extract the largest cluster. Use the box-counting method introduced in the preceding chapter to determine its fractal index. Repeat the procedure at a few other values of p on either side of the percolation threshold, and verify that the fractal dimension of the largest cluster is smallest at $p = p_c$.
5. Generate a 10-member ensemble of 64×64 2-D Cartesian percolation lattices at $p = 0.59$, and build the cluster-size PDF for this data set, using logarithmically constant bin sizes, as described in appendix B.5. Estimate the power-law index by a linear least-squares fit to the logarithm of bin count versus logarithm of size. Now estimate the power-law index (and associated standard error) using the maximum likelihood estimator described in appendix B.6. How well do the two values compare?
6. And now the Grand Challenge! Percolation lattices can be used to study a phenomenon known as *anomalous diffusion*. The idea is as follows: First, generate a 2-D 512×512 lattice at its percolation threshold, identify the largest cluster, and place an ant-like agent (see section 2.4) on an occupied node near the center of this

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

cluster. At each temporal iteration, the ant selects randomly one of the four possible directions top/down/right/left, and steps to that location *only* if the node is occupied; otherwise the ant remains in place until the next temporal iteration. So, in essence, the ant is moving randomly in a "labyrinth" defined by the cluster of which the starting node is part. At each iteration n, calculate the (squared) displacement,

$$D_n^2 = (x_n - x_0)^2 + (y_n - y_0)^2,$$

from the ant's starting position (x_0, y_0). You may let the ant move over a preset number of time steps, but do stop the calculation if the ant reaches the edge of the lattice.

a. Repeat the above simulation process for 10 distinct realizations of your percolation lattice, and plot the ensemble-average rms distance $\sqrt{\langle D_n^2 \rangle}$ versus iteration count.
b. Repeat all of the above for lattices above and below the percolation threshold (at $p = 0.5$ to $p = 0.7$, say).

"Normal" diffusion is characterized by a displacement $\langle d_n \rangle \propto \sqrt{n}$ (see appendix C if needed). In which range of occupation probability p can diffusion be deemed most "anomalous"?

4.8 Further Reading

Much has been written on percolation as an exemplar of criticality. At this writing the classical reference remains

Stauffer, D., and Aharony, A., *Introduction to Percolation Theory*, 2nd ed., Taylor & Francis (1994);

but see also chapter 1 in

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms

Christensen, K., and Moloney, N.R., *Complexity and Criticality*, London: Imperial College Press (2005).

The following offers a grand tour of phase transitions and related behaviors in a variety of physical, biological, and even social systems:

Solé, R.V., *Phase Transitions*, Princeton University Press (2011).

There is also much to be learned from the following book, for those with the appropriate mathematical skills:

Sornette, D., *Critical Phenomena in Natural Sciences*, Springer (2000).

Chapter 12 deals specifically with percolation, but the first four chapters also contain a wealth of useful information on critical systems and the statistical properties of variables distributed as power laws, or other distributions with power-law tails. Algorithms for cluster labeling exist, that are far more efficient than that introduced in section 4.2; see the Stauffer and Aharony book cited above, and also

Newman, M.J.E., and Ziff, R.M., "Efficient Monte Carlo algorithm and high-precision results for percolation," *Phys. Rev. Lett.*, **85**(19), 4104–4107 (2000);

as well as the following Santa Fe Institute working paper by the same authors (viewed June 2016):

http://www.santafe.edu/media/workingpapers/01-02-010.pdf.

On statistically proper techniques for assessing the probability of power-law behavior and determination of their indices from experimental data, see

Clauset, A., Shalizi, C.R., and Newman, M.E.J., "Power-law distributions in empirical data," *SIAM Review*, **51**(4), 661–703.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:54 UTC
All use subject to https://about.jstor.org/terms